Lab 14 — Build & Push a Windows .NET Container Image

K8S-LAB-14 · Windows · Engineer · 75 min

Focus: Dockerfile (Windows), ASP.NET, multi-stage build, ACR/GHCR push, image hygiene

Objective

Author a Windows Dockerfile for an ASP.NET app, build with multi-stage, tag semantically, and push to a registry.

Concepts Covered

• Multi-stage builds on Windows (sdk to runtime)

• ESCAPE directive and PowerShell RUN steps

• Semantic image tagging for Windows artifacts

• Registry auth on Windows (docker login + Credential Manager)

Prerequisites

• Lab 13 complete

• Registry account (ACR, GHCR, or Docker Hub)

Architecture

```
Dev workstation (Win11 + Docker Desktop)
   |  docker build -t myacr.azurecr.io/web:1.0.0 .
   v
+--------+        +---------+
| sdk    | -COPY->| runtime |
| publish|        | aspnet  |
+--------+        +---------+
                       | push
                       v
                  Registry --> Windows worker node
```

Lab Steps

01. Scaffold a minimal ASP.NET app

```
mkdir winweb; cd winweb
dotnet new webapi -n WinWeb -o .
```

Expected: Project files created.

02. Author the Windows Dockerfile

```
# escape=`
FROM mcr.microsoft.com/dotnet/sdk:8.0-windowsservercore-ltsc2022 AS build
WORKDIR C:\src
COPY . .
RUN dotnet publish -c Release -o C:\out

FROM mcr.microsoft.com/dotnet/aspnet:8.0-windowsservercore-ltsc2022
```

```
WORKDIR C:\app
COPY --from=build C:\out .
ENTRYPOINT ["dotnet", "WinWeb.dll"]
```

Expected: Dockerfile written.

03. Build, tag, and inspect

```
docker build -t winweb:1.0.0 .
docker history winweb:1.0.0
```

Expected: Image ~500-700 MB.

04. Push to a registry

```
docker login myacr.azurecr.io
docker tag winweb:1.0.0 myacr.azurecr.io/winweb:1.0.0
docker push myacr.azurecr.io/winweb:1.0.0
```

Expected: Layers uploaded; digest printed.

05. Deploy on the Windows worker

```
kubectl create deploy winweb --image=myacr.azurecr.io/winweb:1.0.0
kubectl patch deploy winweb -p '{"spec":{"template":{"spec":{"nodeSelector":{"kubernetes.io/os":"windows"}}}}}'
```

Expected: Pod scheduled on Windows node, Running.

Common Errors

no matching manifest for windows/amd64
Cause: Pulled Linux-only base image
Fix: Use mcr.microsoft.com/dotnet/* Windows tags

denied: access to resource denied
Cause: Wrong registry login
Fix: docker login + check repo permissions

Massive image size
Cause: Single-stage build keeps SDK
Fix: Multi-stage build

Troubleshooting

• docker build --progress=plain

• trivy image to check CVEs

• Verify base image build matches target nodes

Enterprise Relevance

Windows image hygiene drives cluster cost. Multi-stage + nanoserver where possible is table stakes.

Validation

• Image < 1 GB

• Image runs on Windows node from registry

• No SDK layers in runtime image

Questions

[Beginner] Q: Why multi-stage builds?

A: Strip build-time tooling so runtime image is smaller/safer.

[Intermediate] Q: Why does the Windows Dockerfile use escape directive?

A: Backslash is a path separator; backtick avoids escaping paths.

[Advanced] Q: What is cross-repo mount on push?

A: Registry copies an existing layer blob by digest instead of re-uploading.

[Scenario] Q: CI builds a 6 GB Windows image. Biggest fixes?

A: Multi-stage + switch base to nanoserver if app supports it.

[Viva] Q: Why prefer ACR over Docker Hub for Windows in enterprise?

A: Geo-replication, private endpoint, RBAC, CVE scanning, Azure AD auth.